Fixing Patient-Flow Bottlenecks at Apollo Hospitals

A department-by-department SQL analysis of waiting times, bed utilisation, readmissions and length of stay

SQL Data Analyst project · MySQL 8 · 4 hospitals · 20 departments · 2,500 admissions · 7,300 bed-day records

The problem: patients get stuck in the system

PROBLEM STATEMENT

Apollo Hospitals wants to fix bottlenecks in how patients move through its hospitals: long waits, beds that stay full for too long, and patients getting readmitted.

I was brought in as a SQL Data Analyst to dig into the operational database and find what is actually going wrong, department by department.

4 hospitals	**20** departments
2,500 admissions	**7,300** bed-day records

HOW THIS ANALYSIS IS ORGANISED

1 EXPLORE

Q1: tables, keys and how they connect

2 MEASURE

Q2–Q7: volume, waits, length of stay, readmissions

3 COMPARE & RANK

Q8–Q14: beds, subqueries, CTEs, rankings, top 5

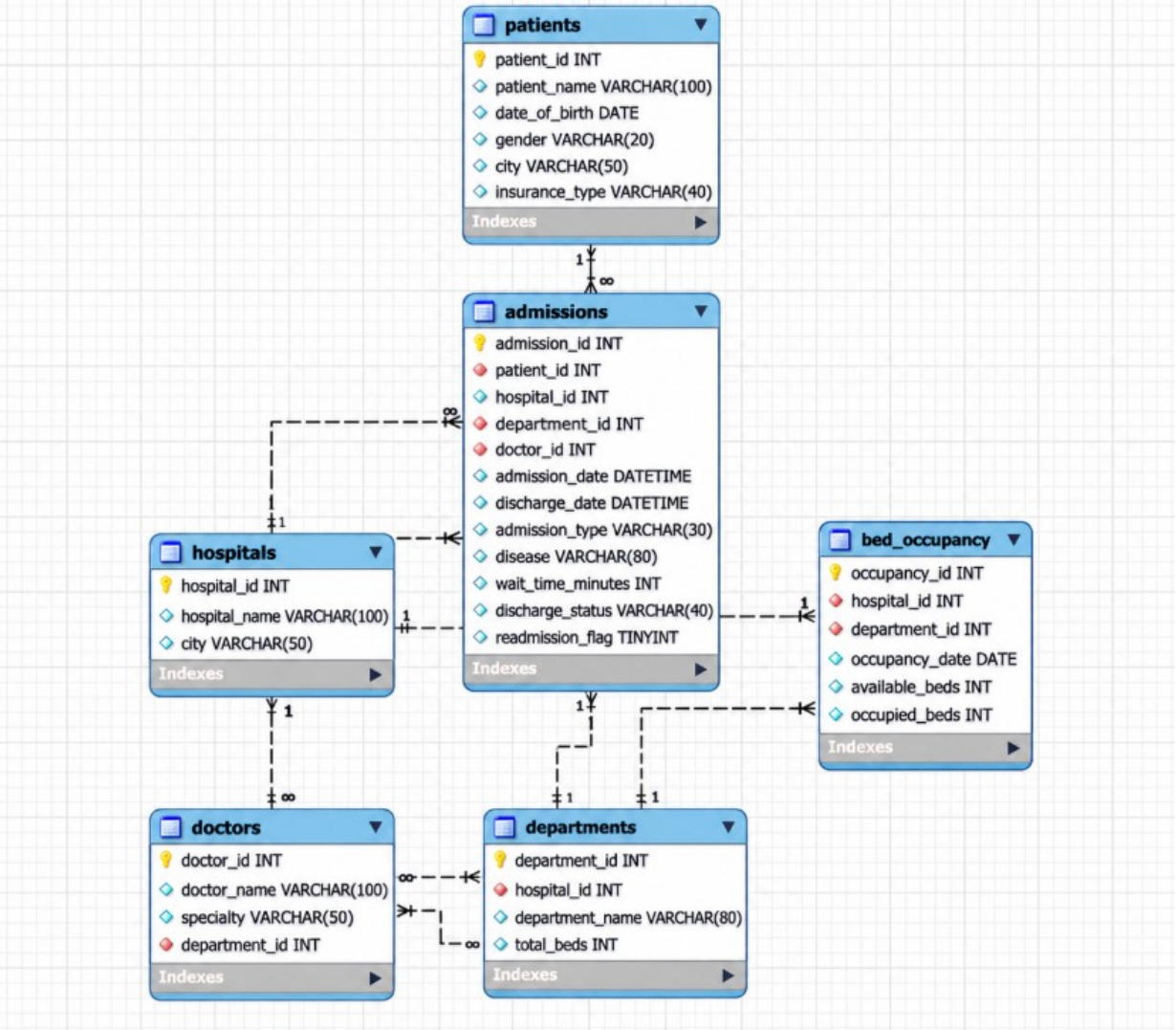
4 OPTIMISE

Q15: make the slowest query fast

THE GOAL

Turn 15 SQL questions into 8 business insights an operations team can act on.

Q1 · Getting to know the database: 6 tables, 8 foreign keys



HOW EVERYTHING CONNECTS

- hospitals → departments → doctors: every hospital has 5 departments (20 in total)
- admissions is the central table: one row per stay, linked to a patient, hospital, department and doctor
- bed_occupancy: one row per department per day (365 × 20 = 7,300 rows)
- Watch-out: admissions and bed_occupancy share only department_id, so joining them directly repeats rows (fixed in Q15)

ROW COUNTS (SQL: DESCRIBE each table + UNION ALL COUNT)

table_name	total_rows
hospitals	4
departments	20
patients	500
doctors	60
admissions	2500
bed_occupancy	7300

TAKEAWAY
Know the keys first, then aggregate big tables separately before joining.

Q2 · Hyderabad is the busiest hospital, but volume is even

PROBLEM
How many admissions has each hospital handled, and which one is busiest?

APPROACH
JOIN admissions to hospitals, COUNT admissions per hospital, sort from highest to lowest.

KEY RESULT
Hyderabad 654 · Mumbai 631 · Delhi 630 · Bangalore 585.

TAKEAWAY
Hospital volume is similar, so the bottleneck is inside departments, not at hospital level.

SQL QUERY

```
SELECT
h.hospital_name,
COUNT(a.admission_id) AS total_admissions
FROM admissions a
JOIN hospitals h ON a.hospital_id = h.hospital_id
GROUP BY h.hospital_id, h.hospital_name
ORDER BY total_admissions DESC;
```

OUTPUT

hospital_name	total_admissions
Apollo Hyderabad	654
Apollo Mumbai	631
Apollo Delhi	630
Apollo Bangalore	585

Q3 · Top 5 departments by patient volume

PROBLEM
Total admissions per department, and the top 5 by patient volume.

APPROACH
GROUP BY department_id (not name, because every hospital has the same department names), ORDER BY count DESC, LIMIT 5.

KEY RESULT
Hyderabad Orthopedics 137 · Hyderabad General Medicine 135 · Delhi General Medicine 134 · Mumbai Cardiology 131 · Hyderabad Emergency 131.

TAKEAWAY
Volume is spread evenly (112 to 137 per department), so volume alone does not explain the bottlenecks.

SQL QUERY

```
select
d.department_name, d.department_id, h.hospital_name,
count(a.admission_id) as total_admissions
from admissions a
join departments d
on d.department_id = a.department_id
join hospitals h
on h.hospital_id = a.hospital_id
group by d.department_name, d.department_id, h.hospital_name
order by total_admissions desc limit 5;
```

OUTPUT

department_name	department_id	hospital_name	total_admissions
Orthopedics	18	Apollo Hyderabad	137
General Medicine	19	Apollo Hyderabad	135
General Medicine	4	Apollo Delhi	134
Emergency	16	Apollo Hyderabad	131
Cardiology	7	Apollo Mumbai	131

Q4 · Emergency departments keep patients waiting longest

PROBLEM
Average, minimum and maximum wait time per department. Which keeps patients waiting longest on average?

APPROACH
AVG, MIN and MAX of wait_time_minutes per department, sorted by the average (the question asks about the average, not the maximum).

KEY RESULT
Bangalore Emergency waits longest: 106.71 min on average. Other Emergency departments: 99.6 to 103.8. General Medicine: about 65.

TAKEAWAY
All four Emergency departments wait about 100 minutes, far above every other department.

SQL QUERY

```
select
h.hospital_name,
d.department_name,
ROUND(AVG(a.wait_time_minutes), 2) AS average_waiting_time,
MIN(a.wait_time_minutes) AS minimum_waiting_time,
MAX(a.wait_time_minutes) AS maximum_waiting_time
FROM admissions a
JOIN departments d ON d.department_id = a.department_id
JOIN hospitals h ON h.hospital_id = d.hospital_id
GROUP BY d.department_id, d.department_name, h.hospital_name
ORDER BY average_waiting_time DESC;
```

OUTPUT

hospital_name	department_name	average_waiting_time	minimum_waiting_time	maximum_waiting_time
Apollo Bangalore	Emergency	106.71	5	217
Apollo Hyderabad	Emergency	103.84	5	221
Apollo Delhi	Emergency	103.78	5	214
Apollo Mumbai	Emergency	99.63	5	204
Apollo Bangalore	General Medicine	66.25	10	125
Apollo Hyderabad	General Medicine	65.75	5	135
Apollo Mumbai	General Medicine	65.37	5	127
Apollo Delhi	General Medicine	64.75	5	122
Apollo Hyderabad	Neurology	51.97	5	116
Apollo Hyderabad	Cardiology	51.56	5	127
Apollo Mumbai	Neurology	48.96	5	96
Apollo Delhi	Orthopedics	48.91	5	96

Q5 · Critical waits (120+ min) are almost all in Emergency

PROBLEM

Bucket patients into Low, Moderate, High and Critical wait categories. Which department has the most Critical cases?

APPROACH

CASE buckets: Low under 30, Moderate 30–59, High 60–119, Critical 120 or more minutes. Then count Critical cases per department.

KEY RESULT

Low 309 · Moderate 1,045 · High 959 · Critical 187. Most Critical: Hyderabad Emergency (46).

TAKEAWAY

179 of the 187 Critical cases sit in the four Emergency departments.

QUERY 1 · Bucket every admission (CASE)

```
select
d.department_id,
d.department_name,
a.admission_id,
a.wait_time_minutes,
case
when wait_time_minutes >= 120 then 'Critical'
when wait_time_minutes >= 60 then 'High'
when wait_time_minutes >= 30 then 'Moderate'
else 'Low'
end as wait_category
from admissions a
join departments d
on d.department_id = a.department_id
order by wait_time_minutes desc;
```

OUTPUT

department_id	department_name	admission_id	wait_time_minutes	wait_category
16	Emergency	1938	221	Critical
16	Emergency	2362	220	Critical
11	Emergency	571	217	Critical
1	Emergency	1482	214	Critical
6	Emergency	2411	204	Critical
11	Emergency	1422	202	Critical
1	Emergency	1937	197	Critical
11	Emergency	2236	197	Critical
16	Emergency	721	194	Critical
6	Emergency	1107	193	Critical
16	Emergency	1601	192	Critical

QUERY 2 · Count Critical cases per department

```
SELECT
h.hospital_name,
d.department_name,
COUNT(*) AS critical_cases
FROM admissions a
JOIN departments d ON d.department_id = a.department_id
JOIN hospitals h ON h.hospital_id = d.hospital_id
WHERE a.wait_time_minutes >= 120
GROUP BY d.department_id, d.department_name, h.hospital_name
ORDER BY critical_cases DESC;
```

OUTPUT

hospital_name	department_name	critical_cases
Apollo Hyderabad	Emergency	46
Apollo Mumbai	Emergency	45
Apollo Bangalore	Emergency	45
Apollo Delhi	Emergency	43
Apollo Delhi	General Medicine	2
Apollo Mumbai	General Medicine	2
Apollo Bangalore	Orthopedics	1
Apollo Bangalore	General Medicine	1
Apollo Hyderabad	Cardiology	1
Apollo Hyderabad	General Medicine	1

Q6 · Delhi Emergency keeps patients longest (4.82 days)

PROBLEM
Average length of stay per department from admission and discharge dates. Which keeps patients longest?

APPROACH
DATEDIFF(discharge_date, admission_date), averaged per department, sorted from longest to shortest.

KEY RESULT
Delhi Emergency 4.82 days · Mumbai Orthopedics 4.76 · Delhi Neurology 4.68. Hospital-wide average: 4.38 days.

TAKEAWAY
Stays are close across departments; Delhi Emergency leads by a small margin.

SQL QUERY

```
select
h.hospital_name,
d.department_name,
round(avg(datediff(a.discharge_date, a.admission_date)),2) as avg_length_of_stay
from admissions a
join departments d
on d.department_id = a.department_id
join hospitals h
on h.hospital_id = d.hospital_id
group by d.department_name, d.department_id, h.hospital_name
order by avg_length_of_stay desc;
```

OUTPUT

hospital_name	department_name	avg_length_of_stay
Apollo Delhi	Emergency	4.82
Apollo Mumbai	Orthopedics	4.76
Apollo Delhi	Neurology	4.68
Apollo Bangalore	Orthopedics	4.63
Apollo Bangalore	Emergency	4.54
Apollo Hyderabad	Emergency	4.53
Apollo Delhi	Orthopedics	4.45
Apollo Delhi	General Medicine	4.41
Apollo Hyderabad	Cardiology	4.39
Apollo Bangalore	Neurology	4.35
Apollo Hyderabad	General Medicine	4.35

Q7 · One in three admissions is a readmission (33.28%)

PROBLEM
Overall readmission rate, and how it compares department to department.

APPROACH
Rate = SUM(readmission_flag) / COUNT(*) × 100, overall and then per department.

KEY RESULT
Overall 33.28% (832 of 2,500). Highest: Delhi Cardiology 40.94%. Lowest: Bangalore General Medicine 27.35%.

TAKEAWAY
A 13-point gap between the best and worst department shows readmissions can be reduced.

QUERY 1 · Overall hospital readmission rate

```
COUNT(*) AS total_admissions,
SUM(readmission_flag) AS total_readmissions,
ROUND(100 * SUM(readmission_flag) / COUNT(*), 2) AS
overall_readmission_rate_pct
FROM admissions;
```

OUTPUT

total_admissions	total_readmissions	overall_readmission_rate_pct
2500	832	33.28

QUERY 2 · Comparing department by department

```
SELECT
h.hospital_name,
d.department_name,
COUNT(*) AS total_admissions,
SUM(a.readmission_flag) AS total_readmissions,
ROUND(100 * SUM(a.readmission_flag) / COUNT(*), 2) AS
readmission_rate_pct
FROM admissions a
JOIN departments d ON d.department_id = a.department_id
JOIN hospitals h ON h.hospital_id = a.hospital_id
GROUP BY d.department_id, d.department_name, h.hospital_name
ORDER BY readmission_rate_pct DESC;
```

OUTPUT

hospital_name	department_name	total_admissions	total_readmissions	readmission_rate_pct
Apollo Delhi	Cardiology	127	52	40.94
Apollo Delhi	Orthopedics	130	53	40.77
Apollo Delhi	Emergency	114	43	37.72
Apollo Hyderabad	Emergency	131	49	37.40
Apollo Hyderabad	Neurology	122	44	36.07
Apollo Delhi	Neurology	125	44	35.20
Apollo Delhi	General Medicine	134	47	35.07
Apollo Bangalore	Cardiology	121	42	34.71
Apollo Bangalore	Neurology	119	41	34.45
Apollo Mumbai	Emergency	122	41	33.61
Apollo Mumbai	Orthopedics	119	38	31.93

Q8 · Repeat patients: 482 of 498 were admitted more than once

PROBLEM
Find patients admitted more than once and rank them by total admissions.

APPROACH
COUNT admissions per patient, HAVING count > 1, DENSE_RANK() so tied patients share a rank.

KEY RESULT
482 of 498 patients came back at least once. Patient 245 has 16 admissions; next highest is 12.

TAKEAWAY
Coming back is the norm: a small group of frequent patients drives a lot of volume.

SQL QUERY

```
SELECT
p.patient_id,
p.patient_name,
COUNT(a.admission_id) AS total_admissions,
DENSE_RANK() OVER (ORDER BY COUNT(a.admission_id) DESC) AS admission_rank
FROM admissions a
JOIN patients p ON p.patient_id = a.patient_id
GROUP BY p.patient_id, p.patient_name
HAVING COUNT(a.admission_id) > 1
ORDER BY admission_rank, p.patient_id
```

OUTPUT

patient_id	patient_name	total_admissions	admission_rank
245	Patient_0245	16	1
266	Patient_0266	12	2
283	Patient_0283	12	2
324	Patient_0324	12	2
331	Patient_0331	12	2
66	Patient_0066	11	3
447	Patient_0447	11	3
30	Patient_0030	10	4
64	Patient_0064	10	4
77	Patient_0077	10	4
99	Patient_0099	10	4

Q9 · Emergency beds run just under the 90% line

PROBLEM
Bed utilisation % per department. Flag anything above 90% as Critical.

APPROACH
SUM(occupied) / SUM(available) × 100 per department over the year, with a CASE flag for above 90%.

KEY RESULT
No department averages above 90%. Emergency is at 88.7 to 89.8% (Bangalore 89.82%); General Medicine about 77%; others 66 to 68%.

TAKEAWAY
Emergency has no buffer: it crosses 90% on 167 to 190 of 365 days.

SQL QUERY

```
select
h.hospital_name,
d.department_name,
round(100 * SUM(b.occupied_beds) / SUM(b.available_beds), 2) as utilisation_pct,
case
when 100 * SUM(b.occupied_beds) / SUM(b.available_beds) > 90 then 'Critical'
else 'Normal'
end as utilisation_status
from bed_occupancy b
join departments d
on d.department_id = b.department_id
join hospitals h
on h.hospital_id = b.hospital_id
group by d.department_name,d.department_id, h.hospital_name
order by utilisation_pct desc;
```

OUTPUT

hospital_name	department_name	utilisation_pct	utilisation_status
Apollo Bangalore	Emergency	89.82	Normal
Apollo Mumbai	Emergency	89.02	Normal
Apollo Delhi	Emergency	89.00	Normal
Apollo Hyderabad	Emergency	88.71	Normal
Apollo Mumbai	General Medicine	77.44	Normal
Apollo Hyderabad	General Medicine	76.96	Normal
Apollo Delhi	General Medicine	76.95	Normal

Q10 · Department report: volume, waiting and beds together

PROBLEM

Build a department-level report with hospital, department, total beds, admissions, average wait and bed utilisation, using multi-table JOINs.

APPROACH

JOIN admissions, departments, hospitals and bed_occupancy. COUNT(DISTINCT) is needed because the bed join repeats every admission.

KEY RESULT

20-row report. Example: Delhi Emergency has 45 beds, 114 admissions, 103.78 min wait and 89.00% utilisation.

TAKEAWAY

One table puts volume, waiting and capacity side by side for each department.

SQL QUERY

```
SELECT
h.hospital_name,
d.department_name,
d.total_beds,
COUNT(DISTINCT a.admission_id) AS total_admissions,
ROUND(AVG(a.wait_time_minutes), 2) AS average_waiting_time,
ROUND(100 * SUM(b.occupied_beds) / SUM(b.available_beds), 2) AS average_bed_utilisation
FROM admissions a
JOIN departments d ON d.department_id = a.department_id
JOIN hospitals h ON h.hospital_id = a.hospital_id
JOIN bed_occupancy b ON b.department_id = a.department_id
GROUP BY d.department_id, h.hospital_name, d.department_name, d.total_beds
ORDER BY h.hospital_name, d.department_name;
```

OUTPUT

hospital_name	department_name	total_beds	total_admissions	average_waiting_time	average_bed_utilisation
Apollo Bangalore	Cardiology	35	121	48.91	65.98
Apollo Bangalore	Emergency	40	116	106.71	89.82
Apollo Bangalore	General Medicine	45	117	66.25	76.55
Apollo Bangalore	Neurology	25	119	47.96	67.38
Apollo Bangalore	Orthopedics	30	112	47.80	66.76
Apollo Delhi	Cardiology	35	127	48.62	66.86
Apollo Delhi	Emergency	45	114	103.78	89.00
Apollo Delhi	General Medicine	40	134	64.75	76.95
Apollo Delhi	Neurology	25	125	47.89	67.38
Apollo Delhi	Orthopedics	30	130	48.91	67.42
Apollo Hyderabad	Cardiology	35	129	51.56	67.11

Q11 · 8 departments wait longer than the hospital-wide average

PROBLEM
Which departments have an average wait time higher than the hospital-wide average? Use a subquery.

APPROACH
A subquery returns the hospital-wide average (62.81 min); HAVING keeps only departments above it.

KEY RESULT
8 departments: all 4 Emergency (99.6 to 106.7 min) and all 4 General Medicine (64.8 to 66.3 min).

TAKEAWAY
Emergency is far above average; General Medicine is only just above it.

SQL QUERY

```
SELECT
h.hospital_name,
d.department_name,
ROUND(AVG(a.wait_time_minutes), 2) AS average_time,
(SELECT ROUND(AVG(wait_time_minutes), 2) FROM admissions) AS hospital_avg
FROM admissions a
JOIN departments d ON d.department_id = a.department_id
JOIN hospitals h ON h.hospital_id = a.hospital_id
GROUP BY d.department_id, d.department_name, h.hospital_name
HAVING AVG(a.wait_time_minutes) > (
SELECT AVG(wait_time_minutes) FROM admissions
)
ORDER BY average_time DESC;
```

OUTPUT

hospital_name	department_name	average_time	hospital_avg
Apollo Bangalore	Emergency	106.71	62.81
Apollo Hyderabad	Emergency	103.84	62.81
Apollo Delhi	Emergency	103.78	62.81
Apollo Mumbai	Emergency	99.63	62.81
Apollo Bangalore	General Medicine	66.25	62.81
Apollo Hyderabad	General Medicine	65.75	62.81
Apollo Mumbai	General Medicine	65.37	62.81
Apollo Delhi	General Medicine	64.75	62.81

Q12 · CTE flags 7 departments that need attention

PROBLEM

Build a CTE with volume, wait, length of stay and readmission rate per department, and flag which need attention.

APPROACH

CTE department_stats holds the 4 measures. Flag = worse than the hospital-wide average on at least 2 of 3 measures (wait, stay, readmission).

KEY RESULT

7 flagged: all 4 Emergency, plus Delhi General Medicine, Delhi Orthopedics and Delhi Neurology.

TAKEAWAY

Delhi has 4 of the 7 flagged departments.

SQL QUERY

```
with department_stats as (
select
h.hospital_name,
d.department_name,
count(a.admission_id) as admission_volume,
round(avg(a.wait_time_minutes),2) as average_wait_time,
round(avg(datediff(a.discharge_date, a.admission_date)),2) as
average_length_of_stay,
ROUND(100 * SUM(a.readmission_flag) / COUNT(a.admission_id), 2) as
readmission_rate_pct
from admissions a
join departments d on d.department_id = a.department_id
join hospitals h on h.hospital_id = a.hospital_id
group by h.hospital_id, h.hospital_name ,d.department_id,
d.department_name
)
select
*,
case
when (average_wait_time > (select AVG(wait_time_minutes) FROM
admissions))
+ ( readmission_rate_pct >(select 100 * SUM(readmission_flag) /
COUNT(*) FROM admissions))
+ ( average_length_of_stay >(select avg(datediff(discharge_date,
admission_date)) from admissions))
>=2
then 'Needs Attention'
else 'Ok'
end as status
from department_stats
order by status, average_wait_time desc;
```

OUTPUT

hospital_name	department_name	admission_volume	average_wait_time	average_length_of_stay	readmission_rate_pct	status
Apollo Bangalore	Emergency	116	106.71	4.54	28.45	Needs Attention
Apollo Hyderabad	Emergency	131	103.84	4.53	37.40	Needs Attention
Apollo Delhi	Emergency	114	103.78	4.82	37.72	Needs Attention
Apollo Mumbai	Emergency	122	99.63	4.27	33.61	Needs Attention
Apollo Delhi	General Medicine	134	64.75	4.41	35.07	Needs Attention
Apollo Delhi	Orthopedics	130	48.91	4.45	40.77	Needs Attention
Apollo Delhi	Neurology	125	47.89	4.68	35.20	Needs Attention
Apollo Bangalore	General Medicine	117	66.25	3.92	27.35	Ok
Apollo Hyderabad	General Medicine	135	65.75	4.35	30.37	Ok
Apollo Mumbai	General Medicine	129	65.37	4.19	31.01	Ok
Apollo Hyderabad	Neurology	122	51.97	4.20	36.07	Ok

Q13 · Ranking departments: the worst one differs by measure

PROBLEM

Rank departments on wait time, readmission rate and bed utilisation.

APPROACH

DENSE_RANK() OVER (ORDER BY measure DESC) for each measure, so rank 1 is the worst and ties share a rank. Two CTEs keep the big tables apart.

KEY RESULT

Bangalore Emergency: rank 1 on wait and on utilisation, but rank 19 on readmissions. Delhi Cardiology: rank 1 on readmissions.

TAKEAWAY

The worst department is different for each measure, so one hospital-wide fix will not work.

SQL QUERY

```
with admission_stats as(
select department_id,
round(avg(wait_time_minutes),2) as avg_wait,
ROUND(100 * SUM(readmission_flag) / COUNT(*), 2) AS readmission_rate
from admissions
group by department_id
),
bed_stats as(
select department_id ,
round(100 * SUM(occupied_beds) / SUM(available_beds), 2) as
utilisation_pct
from bed_occupancy
group by department_id
)
select
h.hospital_name,
d.department_name,
s.avg_wait,
dense_rank() over(order by  s.avg_wait desc ) as wait_rank,
s.readmission_rate,
dense_rank() over(order by s.readmission_rate desc) as
readmission_rank,
b.utilisation_pct,
dense_rank() over(order by b.utilisation_pct desc) as utilisation_rank
from departments d
join hospitals h on h.hospital_id = d.hospital_id
join admission_stats s on s.department_id = d.department_id
join bed_stats b on b.department_id = d.department_id
order by  wait_rank, readmission_rank, utilisation_rank;
```

OUTPUT

hospital_name	department_name	avg_wait	wait_rank	readmission_rate	readmission_rank	utilisation_pct	utilisation_rank
Apollo Bangalore	Emergency	106.71	1	28.45	19	89.82	1
Apollo Hyderabad	Emergency	103.84	2	37.40	4	88.71	4
Apollo Delhi	Emergency	103.78	3	37.72	3	89.00	3
Apollo Mumbai	Emergency	99.63	4	33.61	10	89.02	2
Apollo Bangalore	General Medicine	66.25	5	27.35	20	76.55	8
Apollo Hyderabad	General Medicine	65.75	6	30.37	15	76.96	6
Apollo Mumbai	General Medicine	65.37	7	31.01	13	77.44	5
Apollo Delhi	General Medicine	64.75	8	35.07	7	76.95	7
Apollo Hyderabad	Neurology	51.97	9	36.07	5	66.73	18
Apollo Hyderabad	Cardiology	51.56	10	31.78	12	67.11	15
Apollo Mumbai	Neurology	48.96	11	30.00	16	67.50	11
Apollo Delhi	Orthopedics	48.01	12	40.77	2	67.42	12

Q14 · Top 5 departments with the worst overall patient flow

PROBLEM

Combine wait time, length of stay, readmission rate and bed utilisation into one query that surfaces the top 5 worst departments.

APPROACH

Rank all 4 measures (1 = worst), add them into total_score, sort ascending (lowest total = worst) and take the top 5.

KEY RESULT

Delhi Emergency 10 · Hyderabad Emergency 16 · Bangalore Emergency 26 · Delhi General Medicine 30 · Mumbai Emergency 30.

TAKEAWAY

Delhi Emergency is the clear number one priority: top 3 on three measures and first on length of stay.

SQL QUERY

```
with admission_stats as (
select department_id,
round(avg(wait_time_minutes),2) as avg_wait,
round(avg(datediff(discharge_date, admission_date)),2) as
avg_length_of_stay,
round(100 * SUM(readmission_flag) / COUNT(*), 2) AS readmission_rate
from admissions
group by department_id
),
bed_stats as(
select
department_id,
round(100 * SUM(occupied_beds) / SUM(available_beds), 2) as
utilisation_pct
from bed_occupancy
group by department_id
),
ranked as (
select
h.hospital_name,
d.department_name,
s.avg_wait,
dense_rank() over(order by s.avg_wait desc) as wait_rank,
s.avg_length_of_stay,
dense_rank() over(order by s.avg_length_of_stay desc ) as stay_rank,
s.readmission_rate,
dense_rank() over(order by s.readmission_rate desc ) as
readmission_rank,
b.utilisation_pct,
dense_rank() over(order by b.utilisation_pct desc ) as
utilisation_rank
from departments d
join hospitals h on h.hospital_id = d.hospital_id
join admission_stats s on s.department_id = d.department_id
join bed_stats b on b.department_id = d.department_id
)
select
*,
(wait_rank + stay_rank + readmission_rank + utilisation_rank) as
total_score
from ranked
order by total_score
limit 5;
```

OUTPUT

hospital_name	department_name	avg_wait	wait_rank	avg_length_of_stay	stay_rank	readmission_rate	readmission_rank	utilisation_pct	utilisation_rank	total_score
Apollo Delhi	Emergency	103.78	3	4.82	1	37.72	3	89.00	3	10
Apollo Hyderabad	Emergency	103.84	2	4.53	6	37.40	4	88.71	4	16
Apollo Bangalore	Emergency	106.71	1	4.54	5	28.45	19	89.82	1	26
Apollo Mumbai	Emergency	99.63	4	4.27	14	33.61	10	89.02	2	30
Apollo Delhi	General Medicine	64.75	8	4.41	8	35.07	7	76.95	7	30

Q15 · Query optimisation: from ~905,000 joined rows to 20

BEFORE: the Q10 query joins admissions to bed_occupancy directly

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	h	NULL	ALL	PRIMARY	NULL	NULL	NULL	4	100.00	Using temporary; Using filesort
1	SIMPLE	a	NULL	ref	idx_admissions_hospital,idx_admissions_depart...	idx_admissions_hospital	4	apollo_patient_flow.h.hospital_id	625	100.00	NULL
1	SIMPLE	d	NULL	eq_ref	PRIMARY	PRIMARY	4	apollo_patient_flow.a.department_id	1	100.00	NULL
1	SIMPLE	b	NULL	ref	idx_bed_department_date	idx_bed_department_date	4	apollo_patient_flow.a.department_id	362	100.00	NULL

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EXPLAIN
-> Limit: 5 row(s) (actual time=13747..13747 r...

- Every admission is paired with ~362 bed-day rows: 4 × 625 × 1 × 362 ≈ 905,000 rows
- Plan shows Using temporary; Using filesort
- COUNT(DISTINCT) is needed to undo the repeated rows
- EXPLAIN ANALYZE: 13,747 ms (13.7 s) for 20 result rows

AFTER: summarise each table first, then join 20-row results

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	h	NULL	ALL	PRIMARY	NULL	NULL	NULL	4	100.00	NULL
1	PRIMARY	d	NULL	ref	PRIMARY,hospital_id	hospital_id	4	apollo_patient_flow.h.hospital_id	5	100.00	NULL
1	PRIMARY	<derived2>	NULL	ref	<auto_key0>	<auto_key0>	4	apollo_patient_flow.d.department_id	25	100.00	NULL
1	PRIMARY	<derived3>	NULL	ref	<auto_key0>	<auto_key0>	4	apollo_patient_flow.d.department_id	72	100.00	NULL
3	DERIVED	bed_occupancy	NULL	index	idx_bed_department_date	idx_bed_department_date	7	NULL	7259	100.00	NULL
2	DERIVED	admissions	NULL	index	idx_admissions_department	idx_admissions_department	4	NULL	2500	100.00	NULL

Result Grid | Filter Rows: | Expor

EXPLAIN
-> Nested loop inner join (cost=10043 rows=8...

- Each big table is read once (2,500 and 7,259 rows) inside a CTE
- Joins use 20-row summaries: no temporary table or filesort
- Plain COUNT(*), identical results (2,500 admissions, same averages)
- Indexes were not the problem (every join already used one); the shape of the query was

Metric	Before	After
Rows from bed_occupancy	≈ 905,000 (362 per admission)	7,259, read once
Temporary table + filesort	Yes	No
COUNT(DISTINCT) needed	Yes	No

Lesson: reduce the data first, then join.

Business insights 1–4: wait time, beds and readmissions

INSIGHT 1 · WAIT TIME

FINDING
Emergency is the waiting-time bottleneck.

EVIDENCE
ER average wait is 99.6 to 106.7 min vs 62.8 hospital-wide; 179 of 187 Critical waits (120+ min) are in Emergency.

BUSINESS IMPACT & ACTION
Add a fast-track triage lane and peak-hour ER staffing. Track the % of ER patients waiting over 120 min.

INSIGHT 2 · BED UTILISATION

FINDING
Emergency beds have no surge buffer.

EVIDENCE
ER utilisation is 88.7 to 89.8% for the year and above 90% on 167 to 190 of 365 days.

BUSINESS IMPACT & ACTION
Any spike becomes boarding and longer waits. Set a surge-bed / overflow protocol that triggers at 85% occupancy.

INSIGHT 3 · CAPACITY IMBALANCE

FINDING
Capacity is used unevenly.

EVIDENCE
Cardiology, Neurology and Orthopedics run at 66 to 68% and General Medicine about 77%, vs about 89% in Emergency.

BUSINESS IMPACT & ACTION
Beds sit idle elsewhere while ER is full. Agree flexible beds and an ER-to-ward transfer pathway.

INSIGHT 4 · READMISSIONS

FINDING
One in three admissions is a readmission.

EVIDENCE
33.28% overall (832 of 2,500). Delhi Cardiology 40.94% and Delhi Orthopedics 40.77% vs 27.35% in Bangalore General Medicine.

BUSINESS IMPACT & ACTION
Introduce discharge checklists and 7-day follow-up calls, starting with Delhi Cardiology and Orthopedics; copy Bangalore General Medicine practices.

Business insights 5–8: root causes, stay length and patients

INSIGHT 5 · DIFFERENT ROOT CAUSES

FINDING
Waiting and readmissions are separate problems.

EVIDENCE
Bangalore ER has the longest wait (106.7 min) and fullest beds (89.8%) but a low readmission rate (28.45%, rank 19 of 20).

BUSINESS IMPACT & ACTION
Use flow fixes for Emergency and discharge-quality fixes for Cardiology and Orthopedics, not one hospital-wide programme.

INSIGHT 6 · LENGTH OF STAY

FINDING
Delhi Emergency keeps patients longest.

EVIDENCE
4.82 days vs 4.38 hospital-wide (+10%), with 89% bed utilisation and a 103.8 min wait.

BUSINESS IMPACT & ACTION
Long stays tie up scarce ER beds and feed the queue. Run daily discharge-readiness rounds and track average stay.

INSIGHT 7 · DELHI CONCENTRATION

FINDING
Delhi holds most of the problems.

EVIDENCE
4 of 7 Needs Attention departments are in Delhi; Delhi Emergency scores 10 on the combined ranking (next best 16).

BUSINESS IMPACT & ACTION
Delhi is the best place to pilot the turnaround plan before rolling it out to the other hospitals.

INSIGHT 8 · REPEAT PATIENTS

FINDING
Most patients come back.

EVIDENCE
482 of 498 admitted patients had 2 or more admissions; one patient has 16 (next highest 12).

BUSINESS IMPACT & ACTION
A small group of frequent patients drives volume. Set up care-management plans for patients with 5+ admissions.

What the operations team should do next

#	Action	Where first	KPI to track
1	Fast-track triage lane + peak-hour ER staffing	All 4 Emergency departments (Bangalore and Hyderabad first)	% ER patients waiting over 120 min; ER average wait
2	Surge-bed / overflow protocol at 85% occupancy	Emergency	Days above 90% utilisation (167 to 190 now)
3	Flexible beds + ER-to-ward transfer pathway	Cardiology, Neurology, Orthopedics (about 67% used)	Utilisation gap between ER and wards
4	Discharge checklist + 7-day follow-up call	Delhi Cardiology and Orthopedics	Readmission rate (33.28% baseline)
5	Daily discharge-readiness rounds	Delhi Emergency	Average length of stay (4.82 days baseline)
6	Care-management for frequent patients (5+ admissions)	All hospitals	Share of repeat admissions

NEXT STEP

Pilot the full package at Delhi Emergency (worst combined score), track the KPIs for a quarter, then roll out to the other hospitals. Thank you.